



Experiment 4

Student Name: Prabhakar Kr. Jha

Branch: CSE

Semester: 6th

Subject Name: Full Stack Development – II

UID: 23BCS12284

Section/Group: KRG 3-B

Date of Performance: 03/02/2026

Subject Code: 23CSH-309

1. Aim: To optimize the performance of the EcoTrack React application using **memoization techniques** and **code splitting**, and to enhance the user interface using **enterprise-grade Material UI components**.

2. Objective:

- Understand the causes of unnecessary re-renders in React applications
- Optimize React components using React.memo to prevent avoidable re-renders
- Apply useMemo to efficiently compute derived data and avoid redundant calculations
- Use useCallback to memoize event handler functions and improve component performance
- Implement lazy loading of components and routes using React.lazy and Suspense
- Reduce initial bundle size and improve application load performance through code splitting
- Enhance the visual appearance and usability of the EcoTrack application using Material UI components
- Design a clean, consistent, and responsive user interface using Material UI layouts and typography

3. Implementation / Code:

Tools & Technologies Used:-

- React.js
- React Hooks (useMemo, useCallback)
- React Memo (React.memo)
- Material UI (MUI)
- JavaScript (ES6)
- VS Code
- Web Browser (Google Chrome / Firefox)

Implementation Description:-

- The EcoTrack application is optimized to improve **performance and user experience**.
- **Unnecessary component re-renders** are reduced using React.memo, which ensures components re-render only when their props change.

- The useMemo hook is used to memoize **expensive calculations**, preventing repeated execution on every render.
- The useCallback hook is applied to memoize **event handler functions**, ensuring stable function references across renders.
- Material UI components such as Container, Typography, Button, List, and Divider are used to create a **professional, responsive, and consistent UI**.
- The optimized structure improves application scalability, performance, and maintainability.

Sample Code Snippet:-

```
EcoItem.jsx U X
experiment-4-memoization > ecotrack > src > components > EcoItem.jsx > ...
1  import React from "react";
2  import { ListItem, ListItemText } from "@mui/material";
3
4  const EcoItem = React.memo(({ name }) => {
5    console.log("Item rendered:", name);
6    return (
7      <ListItem>
8        <ListItemText primary={name} />
9      </ListItem>
10    );
11  });
12
13  export default EcoItem;
14
```

```
EcoList.jsx U X
experiment-4-memoization > ecotrack > src > components > EcoList.
1  import { List } from "@mui/material";
2  import EcoItem from "../EcoItem";
3
4  function EcoList({ items }) {
5    return (
6      <List>
7        {items.map((item) => (
8          <EcoItem key={item} name={item} />
9        ))}
10      </List>
11    );
12  }
13
14  export default EcoList;
15
```


